Chain ID: 968 • BOT Testnet

Built with Foundry & OpenZeppelin 5.x

Automated, Trustless Payment Splitting on BOT

Divvy is a serverless, immutable payment splitter protocol for native BOT. Anyone creates a split with percentage cuts for their squad, anyone can pay into it, and each member pulls their share whenever they like.

100%	0%	25 / 25	< N Wei
SERVERLESS & DECENTRALIZED	PROTOCOL FEES TAKEN	PASSING FOUNDRY TESTS	MATHEMATICALLY PROVEN DUST BOUND

Live Contract: 0x4b76926f74117429eb4869f875Cf402BE05997C1 • **Explorer:** https://scan.bohr.life/address/0x4b76926f74117429eb4869f875Cf402BE05997C1

The Problem: Group Payouts in Web3 are Broken

Collaborative squads, hackathon teams, bands, and roommates face high friction and trust barriers when handling shared income.



Manual Math & Spreadsheets

Designating a single "treasurer" to calculate splits and manually distribute crypto results in human errors, calculation disputes, and awkward social follow-ups.



Centralized Custodial Risk

Entrusting group funds to a single personal wallet introduces severe single-point-of-failure risks: loss of keys, exit scams, delays, or unauthorized spending.



Multi-Sig Overhead

Coordinating multi-signature safes for simple revenue sharing requires high operational friction and gas costs, requiring multi-signer quorum for every routine payout.



Push-Payment DoS Vulnerability

Traditional splitter contracts iterate through all members in a single push loop. If one wallet rejects BOT, the whole transaction reverts, freezing everyone's cut.

The Solution: Trustless, Asynchronous Payment Splitting

Divvy replaces manual accounting and multi-sig bureaucracy with a pure, immutable smart contract.

⚡

1. Instant Immutable Split

Name the split (1–50 bytes) and list 2 to 20 member addresses with basis point percentages summing to exactly 10,000 (100.00%). Once created, splits are permanently fixed onchain.

`⚡ Split Equally Auto-Balancing`

📥

2. Permissionless Native Inflow

Clients, sponsors, or smart contracts simply call `pay(id)` with native BOT. Funds accumulate securely inside the contract's verified pool with zero protocol fee deductions.

`Any Payer • Any Increment`

💰

3. Isolated Pull Withdrawals

Each member independently withdraws their entitlement at any time. Owed amounts update dynamically across multiple payments. If a recipient rejects BOT, only their transaction fails.

`Zero Griefing • Strict CEI`

Smart Contract Architecture & Clean Primitives

Built with Solidity ^0.8.24 and OpenZeppelin 5.x with rigorous Checks-Effects-Interactions guarantees.

Robust Onchain Security Design

🔒 **OpenZeppelin ReentrancyGuard:** `nonReentrant` modifiers protect both `pay()` and `withdraw()` entrypoints.

🛡️ **Checks-Effects-Interactions (CEI):** `_released[id][msg.sender]` increments BEFORE low-level external transfers.

🚫 **No receive() / fallback():** Direct plain transfers revert, ensuring all deposits route through `pay(id)` with explicit accounting.

🛑 **Pure Custom Errors:** 12 bespoke custom errors replace heavy revert strings, significantly reducing bytecode size and gas costs.

📜 **Zero Admin Privilege:** No contract owner, no upgrade proxy, no pause switch, and no percentage modification.

Mathematical Precision & Dust Bound

Solidity integer division truncates down, ensuring the contract never pays out more than total received:

```
uint256 totalEntitled = (totalReceived * uint256(bps)) / 10_000;
uint256 owed = totalEntitled - alreadyReleased;
```

Proven Invariant: The dust left behind in the contract is mathematically bounded to be strictly less than the number of members:

Dust ≤ (Member Count - 1) wei < 20 wei

100% Passing Foundry Test Suite (25 / 25)

Exhaustive testing across validation bounds, multi-payment lifecycles, malicious attacks, fuzzing, and invariants.

Unit & Boundary Tests

- ✓ Empty name & >50 chars revert
- ✓ <2 or >20 members revert
- ✓ Array length mismatch revert
- ✓ Zero address & duplicates revert
- ✓ Zero bps & sum ≠ 10,000 revert
- ✓ 0-value & nonexistent split revert

`10 / 10 Passing`

Lifecycle & Edge Cases

- ✓ Multiple separate payments
- ✓ Partial & repeat withdrawals
- ✓ Non-member withdrawal blocked
- ✓ Zero-claimable double-withdraw blocked
- ✓ Awkward amounts (1 wei, 7 wei, 1.000000007 BOT)
- ✓ Direct ETH transfers rejected

`8 / 8 Passing`

Fuzzing & Invariants

- ✓ Malicious reentrant caller blocked
- ✓ Rejecting wallet isolated (others pull cleanly)
- ✓ **Fuzz Test (256 runs):** Zero BOT creation/loss
- ✓ **Invariant Test (640 calls):**

```
balance == totalReceived - totalReleased
```

`7 / 7 Passing`

Modern, Friendly & Responsive Frontend

Built with Vite + React + TypeScript + Viem with mint green and warm yellow visual design.

⚡

Split Builder & Equalizer

- 2 to 20 member input rows with live address validation.
- ⚡ **Split Equally:** Auto-computes equal percentages and puts leftover 0.01% on row 1.
- **Live Total Indicator:** Turns green only at exactly 100.00%, preventing invalid transactions.

👛

Connected "My Splits"

- Instant indexing of all splits where the user's address is listed.
- Highlights percentage share, total received, and claimable balance.
- 1-click **Withdraw** button directly from the dashboard table.

🔗

Shareable Split Hub

- Deep-linking with hash routing (`#/split/<id>`).
- "You" badges identifying the connected user's row in the table.
- Quick-pay chips (+0.1, +0.5, +1.0, +5.0 BOT) for seamless deposits.

Who Uses Divvy? Real-World Target Personas

Any collaborative endeavor in Web3 where revenue, grants, or bills are shared.

💻

Hackathon & Dev Squads

Submit the Divvy split contract as the prize recipient. When sponsors pay out, team members withdraw their agreed shares without waiting on a team captain.

🎸

Bands & Creative Crews

Musicians, digital artists, and podcasters split ticket revenue, NFT mint royalties, and tips proportionally across creators, producers, and managers.

🏠

Roommates & Shared Living

Coliving houses and flatmates pool funds for utilities, rent, and household groceries. Roommates withdraw reimbursements without spreadsheet headaches.

🤖

Dapp & Bot Chains

Autonomous agents and smart contracts on the BOT network route protocol fees directly into Divvy splits for automated operational disbursements.

Roadmap: From Testnet Demo to Ecosystem Standard

A structured progression towards multi-asset payment splitting and developer tooling.

Phase 1: Foundation (Live)

✓ Core Divvy contract deployed on BOT Testnet (Chain 968)
✓ 25/25 passing Foundry test suite with fuzzing & invariants
✓ Responsive Vite + React + TS dapp with hash routing
✓ Equal split generator and instant claim interface

Completed

Phase 2: Mainnet & Presets

- Comprehensive smart contract security audit
- BOT Mainnet deployment and ecosystem listing
- Split presets (50/50, 4-way, custom saved squads)
- Split cloning for recurring projects

Q1 2027

Phase 3: Multi-Token & SDK

- ERC-20 token support (stablecoins & wrapped tokens)
- Automated Split Forwarding between contracts
- "Pay with Divvy" drop-in React/Viem SDK for dapps
- Mobile-optimized PWA with push claim notifications

Q2 2027

Simple. Trustless. Permanent.
Everyone gets their cut.

Network & Contract Coordinates

Network	BOT Testnet (Chain ID: 968)
RPC Endpoint	`https://rpc.bohr.life`
Contract Address	`0x4b76926f74117429eb4869f875Cf402BE05997C1`
Block Explorer	`https://scan.bohr.life`
Frontend App	Running on Vite (Port 5176)

Why Divvy Wins on BOT

✨ **Zero Administrative Barrier:** Splits created in seconds without multi-sig setups.

✨ **Unbreakable Independence:** Each member claims independently with zero blocking risk.

✨ **100% Onchain & Serverless:** No database, no backend API, no custodial keys.

✨ **Mathematical Certainty:** Strict basis point accounting with bounded dust.